

SCINT DETECTION

RegexScintDetector | Pattern Reference

Overview

The `RegexScintDetector` uses pattern matching to identify reality fractures in agent output. This document catalogs the detection patterns.

SYNTAX_TEAR Patterns

Formatting and structure errors.

Severity: 0.3 | Stat: CHA (Charisma/Formatting)

JSON Errors

Python

```
1 # Unclosed braces
2 r'\{[^}]*$'
3
4 # Missing quotes
5 r':\s*[a-zA-Z][a-zA-Z0-9_]*\s*[,}]'
6
7 # Trailing commas
8 r',\s*[}\]]'
9
10 # Invalid escape sequences
11 r'\\[^\\"\\bfnrtu]'
```

XML/HTML Errors

Python

```
1 # Unclosed tags
2 r'<([a-zA-Z]+)[^>]*(?!.*</\1>)'
3
4 # Mismatched tags
5 r'<([a-zA-Z]+)[^>]*.*</(?!\1)[a-zA-Z]+>'
6
7 # Invalid attributes
8 r'<[a-zA-Z]+\s+[^=s>]+\s*(?!=>)'
```

Code Block Errors

Python

```
1 # Unclosed code blocks
2 r'
```

[a-z]n(?!.

```
1  )'  
2  
3  # Mismatched parentheses  
4  r'\([^)]*$|\[^\]]*$'
```

LOGIC_FRACTURE Patterns

Contradictions and paradoxes.

Severity: 0.5 | Stat: INT (Intelligence/Logic)

Mathematical Errors

```
1 # Division by zero
2 r'\s*0(?:[0-9])'
```

```
3
4 # Impossible equations
5 r'\d+\s*[+\.*/]\s*\d+\s*=\s*(?!correct_result)\d+'
6
7 # Contradictory comparisons
8 r'(\d+)\s*>\s*(\d+).*\s*<\s*\d+'
```

Python

Logical Contradictions

```
1 # "X and not X" patterns
2 r'(is|are)\s+(\w+).*(?:is|are)\s+not\s+\d+'
3
4 # "Always" + "Never"
5 r'always.*never|never.*always'
```

```
6
7 # "All" + "None"
8 r'all\s+\w+.*no\s+\w+|no\s+\w+.*all\s+\w+'
```

Python

SAFETY_VOID Patterns

Harmful content indicators.

Severity: 0.9 | Stat: WIS (Wisdom/Safety)

PII Indicators

```
1 # Social Security Numbers
2 r'\b\d{3}-\d{2}-\d{4}\b'
```

```
3
4 # Credit Card Numbers
5 r'\b\d{4}[\s-]?d{4}[\s-]?d{4}[\s-]?d{4}\b'
```

```
6
7 # Email addresses (in certain contexts)
8 r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
```

Python

Harmful Content

```
1 # Explicit refusals (indicating harmful request)
2 r"I(?:cannot|can't|won't|will not) (?:help|assist) with"
```

```
3
4 # Warning phrases
```

Python

```
5 r"(?:dangerous|harmful|illegal|unethical)"
```

HALLUCINATION Patterns

Fabrication indicators.

Severity: 0.6 | **Stat:** INT (Intelligence/Factuality)

Citation Issues

```
1 # Fake URLs
2 r'https?://(?:www\.|)?(?:!known-domains)[a-z]+\.[a-z]+/\S+'
3
4 # Made-up DOIs
5 r'doi:\s*10\.\d{4,}/[^\s]+(?:!verified)'
6
7 # Fabricated years (future)
8 r'\b20[3-9]\d\b|\b2[1-9]\d{2}\b'
```

 Python

Confidence Markers

```
1 # Overconfident hedging
2 r'(?:(?:definitely|certainly|absolutely|100%)\s+(?:maybe|perhaps|might))'
3
4 # False precision
5 r'\b\d+\.\d{5,}\s*%'
```

 Python

Implementing Custom Detectors

```
1 from waft.scint import ScintDetector, ScintType
2
3 class MyDetector(ScintDetector):
4     """Custom Scint detector."""
5
6     def detect(self, text: str) -> list[Scint]:
7         scints = []
8
9         # Your custom patterns
10        if self.my_pattern.search(text):
11            scints.append(Scint(
12                type=ScintType.LOGIC_FRACTURE,
13                severity=0.5,
14                evidence="Custom pattern matched",
15                location=self.get_location(text)
16            ))
17
18        return scints
```

 Python

Severity Calculation

```
1 def calculate_severity(scint: Scint, context: Context) -> float:
2     base = scint.type.base_severity
```

 Python

```
3     difficulty = context.task_complexity / 10
4     history = 1.0 + (context.prior_scints * 0.1)
5
6     return min(1.0, base * difficulty * history)
```

SCINT DETECTION | Know Your Fractures